

Let's Encrypt HTTPS — Status Summary

Revision: 3 **Last modified:** 2026-07-01T11:42:00Z **Status:**
Companion summary of [Status.md](#) (§11.4.56 two-audience).

Page 1 — For the operator / stakeholders (plain language)

We are adding **automatic HTTPS** to the proxy using **Let's Encrypt** — free, auto-renewing TLS certificates — with **Caddy** doing the certificate work in-process (no scripts or cron to babysit) and the **DNS-01** method so it works even without opening ports on the host.

What works today:

- The three setup decisions are locked in (Caddy, DNS-01, test-first then defer the real domain).
- A **certificate checker** is built and proven: it can tell whether a resulting certificate is still valid, how many days are left, whether it covers the right hostname, and whether it was issued by the expected authority — all offline, and it is self-tested so it cannot lie (a deliberately-broken certificate is always caught).
- A **real certificate is now issued end-to-end, fully on this machine** (no internet, no real domain): Caddy asks a local test certificate-authority for a certificate using the DNS method, gets a genuine one, and the checker confirms it is valid, covers the right name, and was really signed by that authority. This runs from scratch on demand in about 15 seconds and produces saved proof each time.
- **Auto-renewal + rotation now works too** (proven on this machine): the system forces the certificate's renewal moment, Caddy obtains a fresh certificate through the normal renewal path (a genuinely new certificate, different serial number), and swaps to it **without dropping a single request** — the

checker confirms the new certificate is valid. So the proxy will keep its HTTPS certificate up to date on its own, with no downtime.

What's still pending / needs you:

- **Going live on the real domain** needs you to provide a DNS provider API token and give the go-ahead. Until then nothing touches production.

Bottom line: the foundation, the safety-checker, real end-to-end certificate issuance, AND automatic zero-downtime renewal are all done and proven on this machine; only the real-domain go-live waits on your go-ahead.

Page 2 — For software engineers

- **Task:** #59 Let's Encrypt HTTPS + auto-renewal + rotation (feature workstream, §11.4.167).
- **Phase 0 (PASS):** operator decisions captured via §11.4.66 — Caddy auto-HTTPS / DNS-01 / hermetic+staging-first, defer real domain. Design LETSENCRYPT_HTTPS.md + plan _PLAN.md @ fddd25c.
- **Phase 1 cert-analyzer (PASS):** tests/letsencrypt/cert_analyzer.sh — 5 pure, now-seam-deterministic (§11.4.50 CERT_ANALYZER_NOW_EPOCH) functions; SAN match is exact-token + single-leading-wildcard (no substring, mirrors _code_in); chain check is issuance-only (openssl verify -no_check_time), orthogonal to expiry. Self-validated §11.4.107(10): cert_analyzer_selftest.sh 37/37; tests/regression/cert_analyzer_selfvalidation_test.sh GREEN+RED_MODE=1 PASS; §1.1 mutation (SAN exact→substring) → guard FAIL, restore byte-identical md5 a06e0f89ee9993fe5de368be852e4eff. Fixtures: public *.pem tracked, *.key/*.csr/*.srl gitignored (§11.4.10), regen via fixtures/gen_fixtures.sh (§11.4.77).
- **Phase 1 custom Caddy image (PASS):** deploy/letsencrypt/build.sh builds localhost/helix_proxy/caddy-challtestsrv:2.8.4 (rootless, non-root user + cap_net_bind_service) embedding the local dns.providers.challtestsrv libdns module via xcaddy; post-build anti-bluff caddy list-modules | grep dns.providers.challtestsrv PASS (§11.4.6 — build ≠ linked).
- **Phase 2 compose + CoreDNS SOA front (PASS):** compose.hermetic.yml — pebble (PEBBLE_VA_ALWAYS_VALID=0) + challtestsrv + **coredns** + caddy; secrets NAME/PATH-only, commented for the hermetic path; caddy loopback-bound (security-audit M1). CoreDNS answers hermetic.test SOA in the ANSWER section (certmagic's zone-determination needs it — challtestsrv NOTIMPs SOA) and fall-through-forwards the dynamic _acme-challenge TXT to challtestsrv.

- **Phase 3 hermetic issuance (PASS):** `deploy/letsencrypt/phase3_hermetic_issue.sh` — clean-slate, re-runnable (§11.4.98): boots the stack, Caddy obtains a REAL cert via DNS-01 from local Pebble (genuine validation), cert-analyzer verifies `not_expired + SAN + negative-SAN-reject + chain-to-this-run's-Pebble-CA`. Two clean runs, evidence in `qa-results/letsencrypt/phase3_issuance/`. Root-cause research for the zone-determination + a certmagic #354 panic in `docs/research/letsencrypt_hermetic_20260701/ + docs/research/certmagic_chain_panic_20260701/`.
- **Phase 5 renewal/rotation (PASS):** `deploy/letsencrypt/phase5_rotation.sh` (re-runnable §11.4.98) — forces the ARI renewal window (storage-surgery on the cached ARI + restart) so Caddy renews via the ACME renewal path to a NEW serial (later `notBefore`); the renewal SWAP is zero-downtime (0 dropped), cert-analyzer verifies the new leaf (chain to per-run Pebble CA). Guard tests `letsencrypt/phase5_rotation_guard.sh` RED+GREEN proven (wired in `run-tests.sh`). The trigger is source-justified (`docs/research/caddy_2110_ari_refetch_20260701/`: certmagic — even $\geq v0.25.1$ /Caddy $\geq 2.11.0$ — has no on-demand ARI re-fetch; production renews on its own schedule). Evidence `qa-results/letsencrypt/phase5_rotation/`.
- **OPERATOR-BLOCKED:** Phase 4 LE-staging (real DNS-01 token §11.4.10), Phase 6 prod cutover (real-domain go-live gate, design §9).
- **Guard wiring:** `cert_analyzer_selfvalidation_test.sh` registered; the Phase-3 issuance guard (`tests/letsencrypt/phase3_issuance_guard.sh`) + LE Challenge land with this lane (§11.4.135).
- **Anti-bluff:** every non-PASS row is honestly PENDING/OPERATOR-BLOCKED (§11.4.6) — no metadata-only PASS; every PASS cites a captured-evidence path.